Harrison Lloyd
CSCI 485
Prof. Shen
4/10/26

Homework 4

Part 1. Data Exploration

1. I loaded the IBM Telco Customer Churn dataset into a pandas DataFrame using pd.read_csv(url). After loading it, I displayed the first few rows with df.head() and confirmed the dataset size using df.shape.
2. The dataset contains 7,043 rows and 21 columns. Each row represents one customer, and each column contains information about customer demographics, services, billing, or churn status. The features are customerID, gender, SeniorCitizen, Partner, Dependents, tenure, PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges, and Churn. The target variable is Churn, which shows whether a customer left the service. Yes means the customer churned, while No means the customer stayed. Most columns had no standard missing values. However, TotalCharges had 11 blank entries, which act like missing values even though they were not first shown as normal NaN values. All other columns had 0 missing values. Because of this, TotalCharges needed extra cleaning before modeling. The target variable is somewhat imbalanced. About 73.46% of customers were in the No class, while about 26.54% were in the Yes class. This means most customers did not churn, but a smaller group still did.
3. This is a classification problem because the target variable, Churn, has two categories, yes/no. The goal is to predict which class a customer belongs to rather than estimate a continuous numeric value. Customer churn prediction is important because losing customers can reduce revenue and increase the cost of acquiring new customers. If a company can identify customers who are at risk of leaving, it can take actions such as better support, targeted offers, or service improvements to increase retention.

Part 2. Data Preprocessing

For preprocessing, I first identified which columns were numerical and which were categorical. The numerical features were SeniorCitizen, tenure, MonthlyCharges, and TotalCharges. The rest of the predictor columns were categorical, such as gender, Partner, Contract, and PaymentMethod. This mattered because numerical and categorical data need

different preprocessing steps. To handle missing values, I found that TotalCharges had 11 blank entries. I converted TotalCharges to numeric using pd.to_numeric(..., errors="coerce"), which turned those invalid blanks into missing values. In the preprocessing pipeline, I used median imputation for numerical columns and most frequent imputation for categorical columns. I chose median for numeric data because it is simple and less affected by outliers, while most frequent imputation works well for filling missing categorical values. I also converted the target variable Churn into binary form by mapping Yes to 1 and No to 0, since the machine learning models require a numeric target. I removed customerID because it is only an identifier and would not help with prediction. For categorical variables, I used one hot encoding with OneHotEncoder(handle_unknown="ignore") so the models could use text based categories as numeric inputs. Finally, I split the dataset into training and test sets using an 80 20 split. I used stratify=y so the churn and non churn class balance would stay similar in both sets. The final shapes were X_train (5634, 19), X_test (1409, 19), y_train (5634,), and y_test (1409,).

```
df_clean = df.copy()

# TODO:
# 1. Drop any unnecessary identifier column(s)
df_clean = df_clean.drop(columns=["customerID"])
# 2. Convert the target column to 0/1
df_clean["Churn"] = df_clean["Churn"].map({"Yes": 1, "No": 0})
# 3. Convert any columns that should be numeric into numeric type
df_clean["TotalCharges"] = pd.to_numeric(df_clean["TotalCharges"], errors="coerce")
```

Part 3. Baseline Model

For the baseline model, I trained a Logistic Regression model using the preprocessing pipeline and training data. I increased max_iter so the model could finish training properly. I then evaluated it using accuracy, precision, recall, F1 score, and ROC AUC. The results were accuracy 0.8077, precision 0.6614, recall 0.5642, F1 score 0.6089, and ROC AUC 0.8421. Overall, the model performed pretty well. Its accuracy was a little over 80 percent, and its ROC AUC was also strong. The recall was lower than the other scores, which means it missed some customers who actually churned. This made Logistic Regression a good baseline to compare with XGBoost.

```
baseline_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=5000))
])

# TODO:
# Fit the baseline model
# Generate predicted labels
# Generate predicted probabilities

# your code here
baseline_model.fit(X_train, y_train)
baseline_preds=baseline_model.predict(X_test)
baseline_probs=baseline_model.predict_proba(X_test)[:, 1]
```

Part 4. XGBoost Model + Hyperparameter Tuning

For this part, I trained an XGBoost classifier using the same preprocessing pipeline. I used GridSearchCV to tune the model and test different values for n_estimators, max_depth, and learning_rate. The values I tested were 50 and 100 for n_estimators, 3 and 5 for max_depth, and 0.05 and 0.1 for learning_rate. I chose these because they affect the number of trees, how complex the trees are, and how quickly the model learns. I used F1 score in grid search because it was my main evaluation metric for this problem. The best hyperparameters were learning_rate = 0.1, max_depth = 5, and n_estimators = 50. The best cross validation F1 score was 0.8074. After tuning, I tested the best XGBoost model on the test set. The results were accuracy 0.7977, precision 0.6459, recall 0.5267, F1 score 0.5803, and ROC AUC 0.8442. These results show that XGBoost performed fairly well, especially on ROC AUC, but its F1 score was lower than the Logistic Regression baseline.

```
xgb_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", XGBClassifier(
        eval_metric="logloss",
        random_state=42
    ))
])

# TODO:
# Define a hyperparameter grid with at least
param_grid = {
    # Example format:
   "classifier__n_estimators": [50, 100],
    "classifier__max_depth": [3, 5],
    "classifier__learning_rate": [0.05, 0.1]
}

param_grid
```

```
grid_search = GridSearchCV(
    estimator=xgb_model,
    param_grid=param_grid,
    scoring=None,    # TODO: replace with
    cv=3,
    n_jobs=-1,
    verbose=1
)

# TODO:
# Fit grid search on training data

# your code here
grid_search.fit(X_train, y_train)
```

Part 5. Choose Evaluation Metric

For this task, I chose F1 score as the main evaluation metric because it balances precision and recall. In churn prediction, both matter. The model should catch customers who are likely to leave, but it also should not label too many loyal customers as churners. F1 score is better than accuracy alone here because the classes are not balanced. Most customers did not churn, while a smaller group did. Because of that, accuracy can look good even if the model mostly predicts the larger class. F1 score gives a better picture of how well the model identifies churners.

Part 6: Feature Importance & Interpretation

Using the trained XGBoost model, I found the top 10 most important features. These were Contract_Month to month, InternetService_Fiber optic, OnlineSecurity_No, InternetService_DSL, TechSupport_No, tenure, Contract_One year, StreamingMovies_Yes, Contract_Two year, and MultipleLines_No. These were the features the model used most when predicting churn. In simple terms, contract type was one of the biggest factors. Customers on month to month contracts seemed more likely to leave than customers on longer contracts. Internet service type also mattered, especially fiber optic. Not having online security or tech support was also important, which suggests those customers may be more likely to churn. Tenure mattered too, showing that how long a customer has stayed with the company plays a role. Overall, the results suggest that customer commitment, service type, and support features are all important in churn.

Part 7: Final Comparison & Reflection

Based on my results, Logistic Regression performed better overall. It had higher accuracy, precision, recall, and F1 score than the tuned XGBoost model. XGBoost was only slightly better on ROC AUC. Since F1 score was my main metric, Logistic Regression was the better final model. XGBoost can still work well because it can learn more complex patterns between features. One limitation is that it is harder to interpret and usually takes more time to tune than Logistic Regression.

Implementation Summary

In this assignment, I used Python in a Jupyter Notebook to analyze the IBM Telco Customer Churn dataset and build two models to predict churn. I started by loading the data and exploring the columns, missing values, and the target variable. Next, I cleaned the data by fixing TotalCharges, converting Churn into 0 and 1, removing customerID, and preparing the numerical and categorical features for modeling. I then split the data into training and test sets. After that, I trained a Logistic Regression model as a baseline and an XGBoost model with hyperparameter tuning. I compared both models using accuracy, precision, recall, F1 score, and ROC AUC. Finally, I looked at feature importance from the XGBoost model to better understand which factors were most related to customer churn.